

Tugas 13: Neural Network Dasar

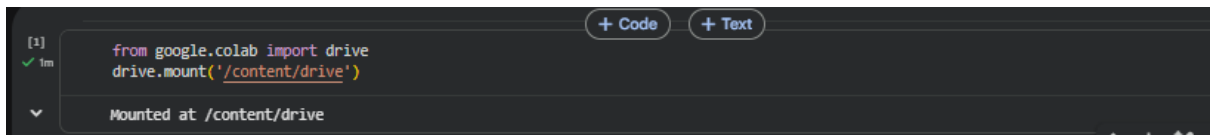
Aisya Az Zahra - 0110224053

Teknik Informatika, STT Terpadu Nurul Fikri, Depok

*E-mail: azzahrasya1705@gmail.com

Abstract. Penelitian ini bertujuan untuk membangun dan mengevaluasi model Artificial Neural Network (ANN) tipe Multilayer Perceptron (MLP) untuk melakukan klasifikasi citra angka tulisan tangan ke dalam sepuluh kelas (0–9). Dataset diproses melalui beberapa tahap preprocessing, meliputi konversi citra ke grayscale, resizing menjadi ukuran 28×28 piksel, normalisasi nilai piksel ke rentang 0–1, serta inversi warna agar sesuai dengan standar dataset MNIST. Data kemudian dibagi menjadi data latih dan data uji dengan perbandingan 80:20 secara stratified. Model MLP dibangun menggunakan dua hidden layer dengan aktivasi ReLU, Batch Normalization, dan Dropout untuk mengurangi overfitting, serta layer output dengan fungsi aktivasi softmax. Proses pelatihan dilakukan menggunakan optimizer Adam dan mekanisme Early Stopping. Hasil evaluasi menunjukkan bahwa model mampu mencapai akurasi pengujian sebesar 90,54%. Evaluasi lebih lanjut menggunakan confusion matrix dan classification report menunjukkan bahwa model memiliki performa yang cukup baik pada seluruh kelas dengan nilai precision, recall, dan f1-score yang relatif seimbang.

1.



```
[1] ✓ 1m
from google.colab import drive
drive.mount('/content/drive')

Mounted at /content/drive
```

Penjelasan :

Kode ini digunakan untuk menghubungkan google colab dengan google drive.

Output:

Output menunjukkan bahwa google colab dengan google drive telah berhasil dijalankan dan telah berhasil terhubung dan kita bisa membuka google colab lewat google drive.

```
[2] ✓ 6s !pip install kaggle
... Requirement already satisfied: kaggle in /usr/local/lib/python3.12/dist-packages (1.7.4.5)
Requirement already satisfied: bleach in /usr/local/lib/python3.12/dist-packages (from kaggle) (6.3.0)
Requirement already satisfied: certifi<14.05.14 in /usr/local/lib/python3.12/dist-packages (from kaggle) (2025.11.12)
Requirement already satisfied: charset-normalizer in /usr/local/lib/python3.12/dist-packages (from kaggle) (3.4.4)
Requirement already satisfied: idna in /usr/local/lib/python3.12/dist-packages (from kaggle) (3.11)
Requirement already satisfied: protobuf in /usr/local/lib/python3.12/dist-packages (from kaggle) (5.29.5)
Requirement already satisfied: python-dateutil<2.5.3 in /usr/local/lib/python3.12/dist-packages (from kaggle) (2.9.0.post0)
Requirement already satisfied: python-slugify in /usr/local/lib/python3.12/dist-packages (from kaggle) (8.0.4)
Requirement already satisfied: requests in /usr/local/lib/python3.12/dist-packages (from kaggle) (2.32.4)
Requirement already satisfied: setuptools<21.0.0 in /usr/local/lib/python3.12/dist-packages (from kaggle) (75.2.0)
Requirement already satisfied: six>=1.10 in /usr/local/lib/python3.12/dist-packages (from kaggle) (1.17.0)
Requirement already satisfied: text-unidecode in /usr/local/lib/python3.12/dist-packages (from kaggle) (1.3)
Requirement already satisfied: tqdm in /usr/local/lib/python3.12/dist-packages (from kaggle) (4.67.1)
Requirement already satisfied: urllib3<1.15.1 in /usr/local/lib/python3.12/dist-packages (from kaggle) (2.5.0)
Requirement already satisfied: webencodings in /usr/local/lib/python3.12/dist-packages (from kaggle) (0.5.1)
```

2.

Penjelasan :

Kode :

Perintah ini digunakan untuk menginstal library Kaggle pada environment Python. Library Kaggle diperlukan untuk mengakses Kaggle API sehingga dataset dapat diunduh langsung dari Kaggle dan digunakan pada proses eksperimen model Artificial Neural Network.

Output :

Output menunjukkan bahwa library Kaggle sudah terpasang pada sistem, sehingga proses instalasi tidak dilakukan kembali. Semua dependensi yang dibutuhkan juga sudah tersedia, menandakan bahwa environment telah siap untuk digunakan dalam proses pengambilan dataset.

```
[3] ✓ 0s kaggle = "/content/drive/MyDrive/Praktikum02/Praktikum13/data/kaggle.json"
```

3.

Penjelasan :

Kode ini digunakan untuk menyimpan lokasi file kaggle.json ke dalam sebuah variabel. File kaggle.json berisi kredensial API Kaggle yang diperlukan untuk melakukan autentikasi saat mengunduh dataset dari Kaggle. Path tersebut menunjukkan bahwa file disimpan di Google Drive agar dapat diakses dari environment Google Colab.

```
[4] ✓ 2s !mkdir -p ~/.kaggle
!cp {kaggle} ~/.kaggle/
!chmod 600 ~/.kaggle/kaggle.json
```

4.

Penjelasan :

- Perintah pertama digunakan untuk membuat folder .kaggle pada direktori home jika folder tersebut belum ada.
- Perintah kedua menyalin file kaggle.json dari lokasi yang telah ditentukan sebelumnya ke folder .kaggle.
- Perintah ketiga mengatur izin akses file kaggle.json agar hanya dapat dibaca oleh pemilik, sesuai dengan ketentuan keamanan Kaggle API.

5.

```
[5] !kaggle datasets download -d olafkrastovski/handwritten-digits-0-9
✓ 1s
Dataset URL: https://www.kaggle.com/datasets/olafkrastovski/handwritten-digits-0-9
License(s): CC0-1.0
Downloading handwritten-digits-0-9.zip to /content
 0% 0.00/66.7M [00:00<?, ?B/s]
100% 66.7M/66.7M [00:00<00:00, 1.38GB/s]
```

Penjelasan :

Kode :

Perintah ini digunakan untuk mengunduh dataset handwritten-digits-0-9 dari Kaggle menggunakan Kaggle API. Dataset ini berisi citra angka tulisan tangan dari kelas 0 sampai 9 yang akan digunakan sebagai data input pada model Artificial Neural Network untuk tugas klasifikasi multi-kelas.

Output :

Output menunjukkan informasi dataset yang diunduh beserta lisensinya. Proses pengunduhan berhasil dilakukan dengan ukuran file sekitar 66.7 MB dan disimpan dalam bentuk file ZIP di direktori /content. Hal ini menandakan dataset telah siap untuk diekstrak dan diproses lebih lanjut.

6.

```
[6] from zipfile import ZipFile
✓ 8s import os
file_name = '/content/handwritten-digits-0-9.zip'

extract_folder = "dataset"
os.makedirs(extract_folder, exist_ok=True)

with ZipFile(file_name, 'r') as zip:
    zip.extractall(extract_folder)
    print('Done')

Done
```

Penjelasan :

Kode :

Kode ini digunakan untuk mengekstrak file dataset yang masih berbentuk ZIP. Library ZipFile digunakan untuk membuka dan mengekstrak isi file, sedangkan library os digunakan untuk membuat folder tujuan penyimpanan hasil ekstraksi. Folder dataset dibuat terlebih dahulu jika belum ada, kemudian seluruh isi file ZIP diekstrak ke dalam folder tersebut.

Output :

Output “Done” menandakan bahwa proses ekstraksi file ZIP telah berhasil dilakukan. Seluruh file dataset kini tersedia dalam folder dataset dan siap digunakan pada tahap preprocessing data citra.

7.

```
[7]
✓ 0s
data_0 = os.listdir('/content/dataset/0')
data_1 = os.listdir('/content/dataset/1')
data_2 = os.listdir('/content/dataset/2')
data_3 = os.listdir('/content/dataset/3')
data_4 = os.listdir('/content/dataset/4')
data_5 = os.listdir('/content/dataset/5')
data_6 = os.listdir('/content/dataset/6')
data_7 = os.listdir('/content/dataset/7')
data_8 = os.listdir('/content/dataset/8')
data_9 = os.listdir('/content/dataset/9')
```

Penjelasan :

Kode ini digunakan untuk mengambil daftar nama file citra pada setiap folder kelas digit dari 0 sampai 9. Setiap folder merepresentasikan satu kelas label, sehingga daftar file ini akan digunakan untuk mengetahui jumlah data pada masing-masing kelas dan sebagai dasar dalam proses pemuatan serta pelabelan data citra.

8.

```
[8]
✓ 0s
len(data_0)
2236
```

Penjelasan :

Kode :

Kode ini digunakan untuk menghitung jumlah file citra yang terdapat pada folder kelas digit 0. Fungsi len() menghitung banyaknya elemen dalam daftar data_0, yang sebelumnya berisi nama-nama file citra digit 0.

Output :

Output menunjukkan bahwa terdapat 2236 citra angka tulisan tangan untuk kelas digit 0. Informasi ini digunakan untuk mengetahui distribusi data pada setiap kelas dan memastikan bahwa dataset memiliki jumlah data yang memadai untuk proses training dan evaluasi model.

9.

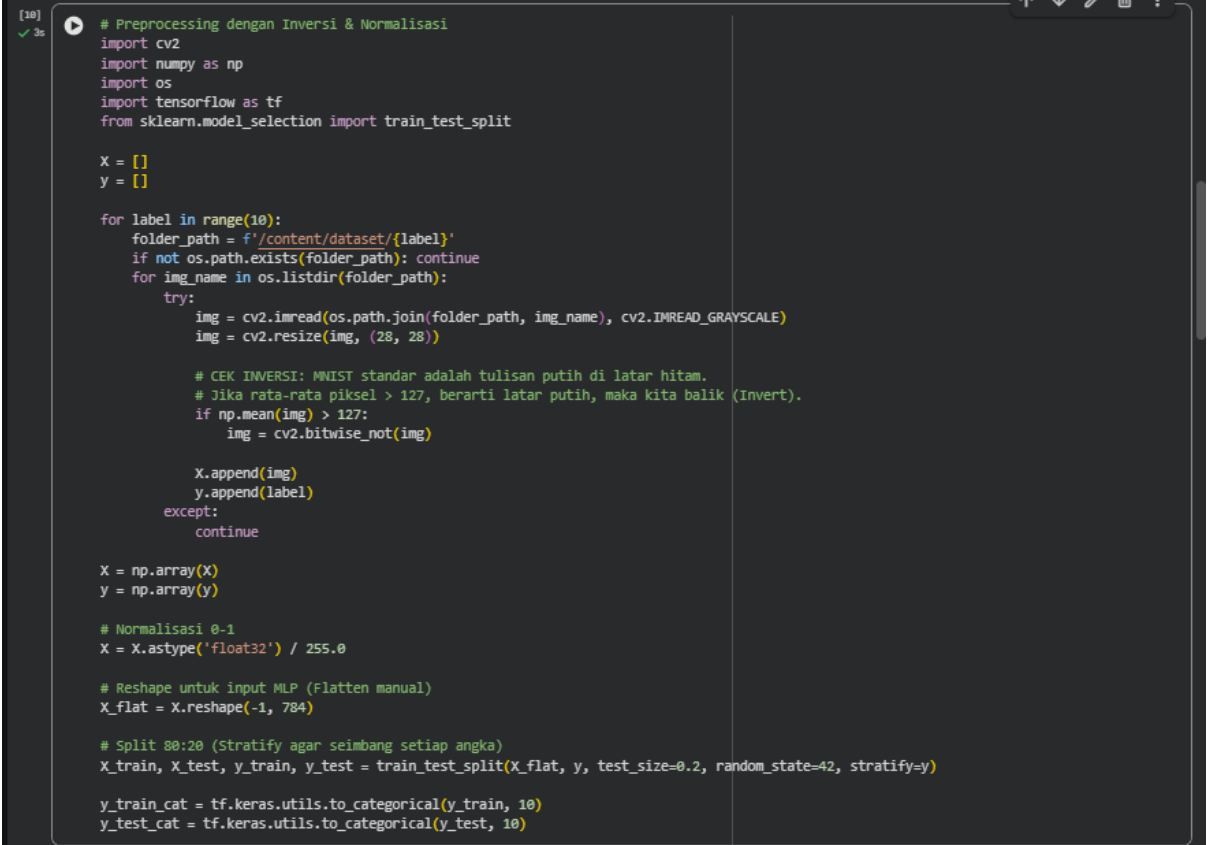
```
[9]
✓ 17s
# IMPORT LIBRARY
import os
import cv2
import numpy as np
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout, Flatten, BatchNormalization
from tensorflow.keras.callbacks import EarlyStopping
from sklearn.model_selection import train_test_split
from sklearn.metrics import confusion_matrix, classification_report
import matplotlib.pyplot as plt
import seaborn as sns
```

Penjelasan :

Kode ini digunakan untuk mengimpor seluruh library yang dibutuhkan dalam pembuatan dan evaluasi model Artificial Neural Network.

- Library os digunakan untuk pengelolaan file dan direktori
- cv2 untuk membaca dan memproses citra

- numpy untuk pengolahan data numerik
- TensorFlow dan Keras digunakan untuk membangun, melatih, dan mengevaluasi model MLP
- Library scikit-learn digunakan untuk membagi data dan menghitung metrik evaluasi
- matplotlib dan seaborn digunakan untuk visualisasi hasil seperti confusion matrix.

10. 

```
[10]
✓ 2s
# Preprocessing dengan Inversi & Normalisasi
import cv2
import numpy as np
import os
import tensorflow as tf
from sklearn.model_selection import train_test_split

X = []
y = []

for label in range(10):
    folder_path = f'/content/dataset/{label}'
    if not os.path.exists(folder_path): continue
    for img_name in os.listdir(folder_path):
        try:
            img = cv2.imread(os.path.join(folder_path, img_name), cv2.IMREAD_GRAYSCALE)
            img = cv2.resize(img, (28, 28))

            # CEK INVERSI: MNIST standar adalah tulisan putih di latar hitam.
            # Jika rata-rata piksel > 127, berarti latar putih, maka kita balik (Invert).
            if np.mean(img) > 127:
                img = cv2.bitwise_not(img)

            X.append(img)
            y.append(label)
        except:
            continue

X = np.array(X)
y = np.array(y)

# Normalisasi 0-1
X = X.astype('float32') / 255.0

# Reshape untuk input MLP (Flatten manual)
X_flat = X.reshape(-1, 784)

# Split 80:20 (Stratify agar seimbang setiap angka)
X_train, X_test, y_train, y_test = train_test_split(X_flat, y, test_size=0.2, random_state=42, stratify=y)

y_train_cat = tf.keras.utils.to_categorical(y_train, 10)
y_test_cat = tf.keras.utils.to_categorical(y_test, 10)
```

Penjelasan :

Kode ini melakukan preprocessing data citra angka tulisan tangan. Setiap citra dibaca dalam format grayscale dan diubah ukurannya menjadi 28x28 piksel agar sesuai dengan format MNIST. Selanjutnya dilakukan pengecekan inversi warna, di mana citra dengan latar putih dan tulisan gelap dibalik agar menyerupai standar MNIST, yaitu tulisan putih dengan latar hitam. Data citra dan label kemudian disimpan dalam array, dinormalisasi ke rentang 0–1, dan diubah menjadi bentuk vektor satu dimensi untuk digunakan sebagai input model MLP. Dataset kemudian dibagi menjadi data latih dan data uji dengan rasio 80:20 secara seimbang, serta label dikonversi ke format one-hot encoding.

```
[11] ✓ 0s
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout, BatchNormalization
from tensorflow.keras.optimizers import Adam

model = Sequential([
    # Hidden Layer 1
    Dense(512, activation='relu', input_shape=(784,)),
    BatchNormalization(),
    Dropout(0.2),

    # Hidden Layer 2
    Dense(256, activation='relu'),
    BatchNormalization(),
    Dropout(0.2),

    # Output Layer
    Dense(10, activation='softmax')
])

# Gunakan Learning Rate yang lebih kecil (0.0001) agar belajar lebih teliti
model.compile(optimizer=Adam(learning_rate=0.0001),
              loss='categorical_crossentropy',
              metrics=['accuracy'])

/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:93: UserWarning: Do not pass an `input_shape` / `input_dim` argument to
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

11.

Penjelasan :

Kode :

Kode ini digunakan untuk membangun arsitektur model Multi-Layer Perceptron menggunakan Keras Sequential. Model terdiri dari dua hidden layer dengan masing-masing 512 dan 256 neuron menggunakan fungsi aktivasi ReLU. Batch Normalization digunakan untuk menstabilkan proses training, sedangkan Dropout digunakan untuk mengurangi risiko overfitting. Layer output menggunakan 10 neuron dengan fungsi aktivasi softmax karena model melakukan klasifikasi multi-kelas digit 0 sampai 9. Model kemudian dikompilasi menggunakan optimizer Adam dengan learning rate kecil agar proses pembelajaran lebih stabil, serta menggunakan fungsi loss categorical crossentropy dan metrik akurasi.

Output :

Output berupa peringatan yang menyatakan bahwa penggunaan parameter input_shape secara langsung pada layer Dense tidak direkomendasikan pada model Sequential versi Keras terbaru. Peringatan ini tidak menghentikan eksekusi program dan model tetap dapat berjalan dengan normal. Peringatan tersebut hanya bersifat saran agar input didefinisikan menggunakan layer Input pada awal model.

```
[12] ✓ 0s
# --- NORMALISASI (Kunci Akurasi Tinggi) ---
# Mengubah rentang 0-255 menjadi 0-1
X = X.astype('float32') / 255.0
```

12.

Penjelasan :

Kode ini digunakan untuk melakukan normalisasi nilai piksel citra dari rentang 0–255 menjadi 0–1. Normalisasi dilakukan dengan mengubah tipe data menjadi float32 kemudian membagi setiap nilai piksel dengan 255. Proses ini bertujuan agar data memiliki skala yang seragam sehingga proses training model MLP menjadi lebih stabil dan cepat konvergen.

13.

```
[13]
✓ 0s # Reshape untuk MLP (Flat)
X_flat = X.reshape(-1, 28*28)
```

Penjelasan :

Kode ini digunakan untuk mengubah bentuk data citra dari matriks dua dimensi berukuran 28×28 menjadi vektor satu dimensi dengan panjang 784. Proses reshape ini diperlukan karena model Multi-Layer Perceptron menerima input dalam bentuk vektor, bukan citra dua dimensi.

14.

```
[14]
✓ 0s # Split 80:20
X_train, X_test, y_train, y_test = train_test_split(X_flat, y, test_size=0.2, random_state=42, stratify=y)

y_train_cat = tf.keras.utils.to_categorical(y_train, 10)
y_test_cat = tf.keras.utils.to_categorical(y_test, 10)
```

Penjelasan :

Kode ini digunakan untuk membagi dataset menjadi data latih dan data uji dengan rasio 80:20. Parameter stratify digunakan agar distribusi kelas digit pada data latih dan data uji tetap seimbang. Setelah itu, label kelas diubah ke dalam bentuk one-hot encoding agar sesuai dengan output model MLP yang menggunakan fungsi aktivasi softmax dan loss categorical crossentropy.

15.

```
[15]
✓ 0s from tensorflow.keras.callbacks import EarlyStopping
import numpy as np
from sklearn.metrics import classification_report, accuracy_score

# Konfigurasi Early Stopping
early_stop = EarlyStopping(
    monitor='val_loss',
    patience=10,
    restore_best_weights=True, # Mengembalikan bobot terbaik ke model setelah selesai
    verbose=1
)
```

Penjelasan :

Kode ini digunakan untuk mengonfigurasi callback Early Stopping pada proses training model. Early Stopping memantau nilai validation loss dan akan menghentikan training apabila tidak terjadi perbaikan selama 10 epoch berturut-turut. Opsi restore_best_weights digunakan agar model kembali menggunakan bobot terbaik yang diperoleh selama training, sehingga performa model menjadi optimal dan terhindar dari overfitting.

16.

```
[16]
✓ 5m # Proses Training
print("Memulai proses training...")
history = model.fit(
    X_train, y_train_cat,
    epochs=100,
    batch_size=32,
    validation_data=(X_test, y_test_cat),
    callbacks=[early_stop],
    verbose=1
)
```

Epoch 23/100
539/539 ————— 7s 14ms/step - accuracy: 0.9026 - loss: 0.3302 - val_accuracy: 0.8812 - val_loss: 0.4166
Epoch 24/100
539/539 ————— 6s 11ms/step - accuracy: 0.8975 - loss: 0.3327 - val_accuracy: 0.8766 - val_loss: 0.4134
Epoch 25/100
539/539 ————— 7s 14ms/step - accuracy: 0.9104 - loss: 0.2995 - val_accuracy: 0.8877 - val_loss: 0.3816
Epoch 26/100
539/539 ————— 6s 11ms/step - accuracy: 0.9104 - loss: 0.2926 - val_accuracy: 0.8822 - val_loss: 0.3975
Epoch 27/100
539/539 ————— 8s 15ms/step - accuracy: 0.9161 - loss: 0.2753 - val_accuracy: 0.8910 - val_loss: 0.3762
Epoch 28/100
539/539 ————— 8s 11ms/step - accuracy: 0.9139 - loss: 0.2764 - val_accuracy: 0.8896 - val_loss: 0.3775
Epoch 29/100
539/539 ————— 10s 11ms/step - accuracy: 0.9166 - loss: 0.2668 - val_accuracy: 0.8898 - val_loss: 0.3762
Epoch 30/100
539/539 ————— 7s 12ms/step - accuracy: 0.9193 - loss: 0.2651 - val_accuracy: 0.8900 - val_loss: 0.3775
Epoch 31/100
539/539 ————— 6s 12ms/step - accuracy: 0.9232 - loss: 0.2451 - val_accuracy: 0.8840 - val_loss: 0.3957
Epoch 32/100
539/539 ————— 7s 12ms/step - accuracy: 0.9225 - loss: 0.2425 - val_accuracy: 0.8982 - val_loss: 0.3569
Epoch 33/100
539/539 ————— 9s 11ms/step - accuracy: 0.9260 - loss: 0.2364 - val_accuracy: 0.9000 - val_loss: 0.3487
Epoch 34/100
539/539 ————— 7s 14ms/step - accuracy: 0.9281 - loss: 0.2276 - val_accuracy: 0.8986 - val_loss: 0.3492
Epoch 35/100
539/539 ————— 8s 14ms/step - accuracy: 0.9316 - loss: 0.2167 - val_accuracy: 0.8984 - val_loss: 0.3615
Epoch 36/100
539/539 ————— 7s 14ms/step - accuracy: 0.9381 - loss: 0.2084 - val_accuracy: 0.8942 - val_loss: 0.3585
Epoch 37/100
539/539 ————— 6s 11ms/step - accuracy: 0.9351 - loss: 0.2085 - val_accuracy: 0.8982 - val_loss: 0.3621
Epoch 38/100
539/539 ————— 11s 21ms/step - accuracy: 0.9446 - loss: 0.1894 - val_accuracy: 0.9016 - val_loss: 0.3499
Epoch 39/100
539/539 ————— 6s 12ms/step - accuracy: 0.9397 - loss: 0.1957 - val_accuracy: 0.8924 - val_loss: 0.3758
Epoch 40/100
539/539 ————— 7s 13ms/step - accuracy: 0.9408 - loss: 0.1914 - val_accuracy: 0.9054 - val_loss: 0.3292
Epoch 41/100
539/539 ————— 7s 12ms/step - accuracy: 0.9384 - loss: 0.1911 - val_accuracy: 0.8952 - val_loss: 0.3796
Epoch 42/100

```

539/539 ----- 7s 14ms/step - accuracy: 0.9020 - loss: 0.2502 - val_accuracy: 0.8812 - val_loss: 0.4100
Epoch 24/100
539/539 ----- 6s 11ms/step - accuracy: 0.8975 - loss: 0.3327 - val_accuracy: 0.8766 - val_loss: 0.4134
Epoch 25/100
539/539 ----- 7s 14ms/step - accuracy: 0.9104 - loss: 0.2995 - val_accuracy: 0.8877 - val_loss: 0.3816
Epoch 26/100
539/539 ----- 6s 11ms/step - accuracy: 0.9104 - loss: 0.2926 - val_accuracy: 0.8822 - val_loss: 0.3975
Epoch 27/100
539/539 ----- 8s 15ms/step - accuracy: 0.9161 - loss: 0.2753 - val_accuracy: 0.8910 - val_loss: 0.3762
Epoch 28/100
539/539 ----- 8s 11ms/step - accuracy: 0.9139 - loss: 0.2764 - val_accuracy: 0.8896 - val_loss: 0.3775
Epoch 29/100
539/539 ----- 10s 11ms/step - accuracy: 0.9166 - loss: 0.2668 - val_accuracy: 0.8898 - val_loss: 0.3762
Epoch 30/100
539/539 ----- 7s 12ms/step - accuracy: 0.9193 - loss: 0.2651 - val_accuracy: 0.8900 - val_loss: 0.3775
Epoch 31/100
539/539 ----- 6s 12ms/step - accuracy: 0.9232 - loss: 0.2451 - val_accuracy: 0.8840 - val_loss: 0.3957
Epoch 32/100
539/539 ----- 7s 12ms/step - accuracy: 0.9225 - loss: 0.2425 - val_accuracy: 0.8982 - val_loss: 0.3569
Epoch 33/100
539/539 ----- 9s 11ms/step - accuracy: 0.9260 - loss: 0.2364 - val_accuracy: 0.9000 - val_loss: 0.3487
Epoch 34/100
539/539 ----- 7s 14ms/step - accuracy: 0.9281 - loss: 0.2276 - val_accuracy: 0.8986 - val_loss: 0.3492
Epoch 35/100
539/539 ----- 8s 14ms/step - accuracy: 0.9316 - loss: 0.2167 - val_accuracy: 0.8984 - val_loss: 0.3615
Epoch 36/100
539/539 ----- 7s 14ms/step - accuracy: 0.9381 - loss: 0.2084 - val_accuracy: 0.8942 - val_loss: 0.3585
Epoch 37/100
539/539 ----- 6s 11ms/step - accuracy: 0.9351 - loss: 0.2085 - val_accuracy: 0.8982 - val_loss: 0.3621
Epoch 38/100
539/539 ----- 11s 21ms/step - accuracy: 0.9446 - loss: 0.1894 - val_accuracy: 0.9016 - val_loss: 0.3499
Epoch 39/100
539/539 ----- 6s 12ms/step - accuracy: 0.9397 - loss: 0.1957 - val_accuracy: 0.8924 - val_loss: 0.3758
Epoch 40/100
539/539 ----- 7s 13ms/step - accuracy: 0.9408 - loss: 0.1914 - val_accuracy: 0.9054 - val_loss: 0.3292
Epoch 41/100
539/539 ----- 7s 12ms/step - accuracy: 0.9384 - loss: 0.1911 - val_accuracy: 0.8952 - val_loss: 0.3796
Epoch 42/100
539/539 ----- 7s 13ms/step - accuracy: 0.9393 - loss: 0.1888 - val_accuracy: 0.8979 - val_loss: 0.3585
Epoch 43/100
539/539 ----- 6s 11ms/step - accuracy: 0.9446 - loss: 0.1763 - val_accuracy: 0.9021 - val_loss: 0.3454
Epoch 44/100
539/539 ----- 7s 14ms/step - accuracy: 0.9426 - loss: 0.1826 - val_accuracy: 0.9010 - val_loss: 0.3472
Epoch 45/100
539/539 ----- 10s 12ms/step - accuracy: 0.9486 - loss: 0.1650 - val_accuracy: 0.9049 - val_loss: 0.3394
Epoch 46/100
539/539 ----- 6s 12ms/step - accuracy: 0.9482 - loss: 0.1634 - val_accuracy: 0.9054 - val_loss: 0.3342
Epoch 47/100
539/539 ----- 6s 11ms/step - accuracy: 0.9488 - loss: 0.1662 - val_accuracy: 0.9121 - val_loss: 0.3318
Epoch 48/100
539/539 ----- 7s 13ms/step - accuracy: 0.9505 - loss: 0.1558 - val_accuracy: 0.8996 - val_loss: 0.3423
Epoch 49/100
539/539 ----- 6s 11ms/step - accuracy: 0.9515 - loss: 0.1474 - val_accuracy: 0.9042 - val_loss: 0.3577
Epoch 50/100
539/539 ----- 7s 14ms/step - accuracy: 0.9527 - loss: 0.1502 - val_accuracy: 0.9040 - val_loss: 0.3427
Epoch 50: early stopping
Restoring model weights from the end of the best epoch: 40.

```

Penjelasan :

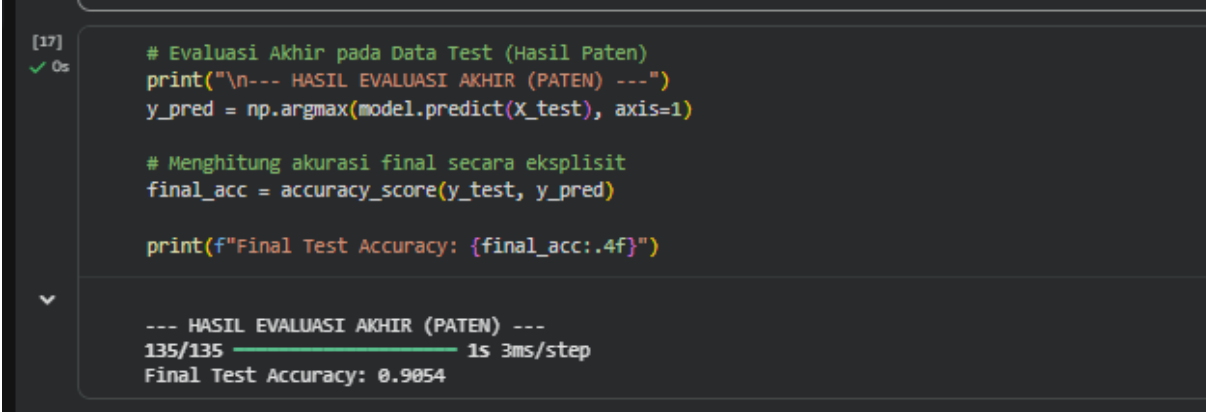
Kode :

Kode ini digunakan untuk melatih model MLP menggunakan data latih yang telah dipreprocessing. Model dilatih selama maksimal 100 epoch dengan ukuran batch 32. Data uji digunakan sebagai data validasi untuk memantau performa model selama training. Callback Early Stopping diterapkan untuk menghentikan training secara otomatis apabila nilai validation loss tidak mengalami perbaikan, sehingga dapat mencegah overfitting dan menghemat waktu komputasi.

Output :

Output menunjukkan proses training model dari epoch awal hingga epoch ke-50. Selama training, nilai accuracy dan loss pada data latih serta data validasi terus mengalami peningkatan. Proses training dihentikan secara otomatis pada epoch ke-50 karena tidak terjadi perbaikan validation loss sesuai dengan konfigurasi Early Stopping. Model

kemudian menggunakan bobot terbaik yang diperoleh pada epoch ke-40, yang memberikan performa validasi terbaik.

17. 

```
[17] ✓ 0s # Evaluasi Akhir pada Data Test (Hasil Paten)
print("\n--- HASIL EVALUASI AKHIR (PATEN) ---")
y_pred = np.argmax(model.predict(X_test), axis=1)

# Menghitung akurasi final secara eksplisit
final_acc = accuracy_score(y_test, y_pred)

print(f"Final Test Accuracy: {final_acc:.4f}")

--- HASIL EVALUASI AKHIR (PATEN) ---
135/135 ————— 1s 3ms/step
Final Test Accuracy: 0.9054
```

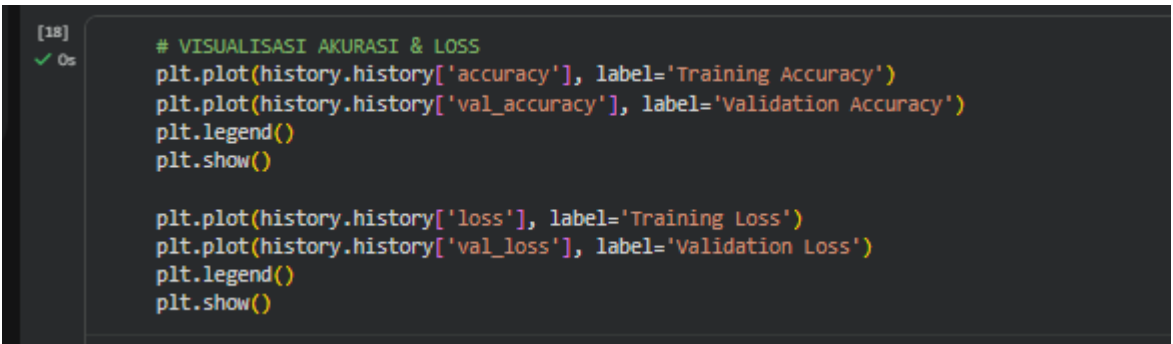
Penjelasan :

Kode :

Kode ini digunakan untuk melakukan evaluasi akhir model MLP pada data uji. Model menghasilkan prediksi probabilitas untuk setiap kelas digit, kemudian nilai dengan probabilitas tertinggi dipilih sebagai label prediksi. Selanjutnya, akurasi akhir dihitung secara eksplisit dengan membandingkan label prediksi dan label sebenarnya menggunakan fungsi `accuracy_score`.

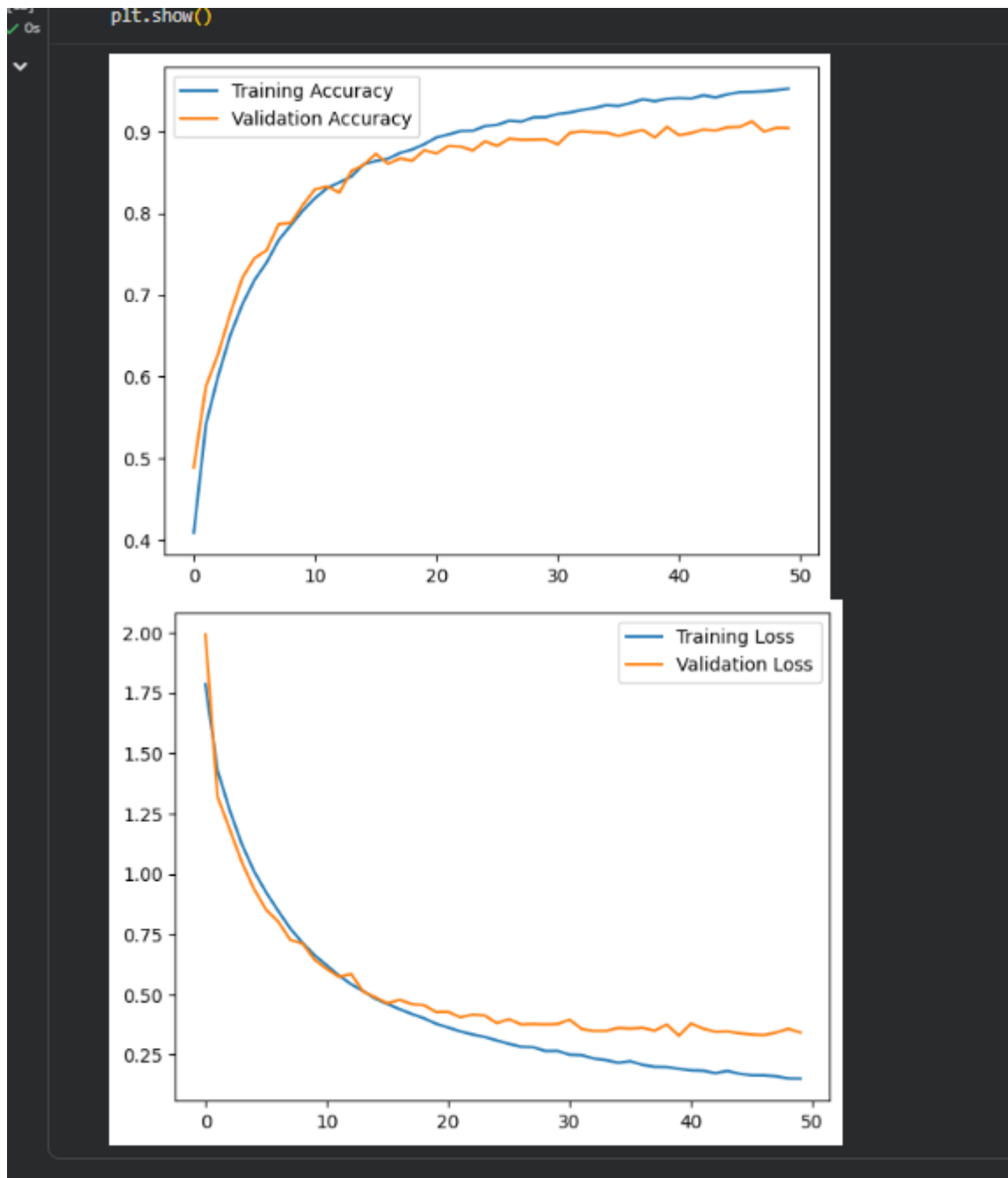
Output :

Output menunjukkan bahwa proses prediksi pada data uji telah selesai dilakukan. Nilai akurasi akhir sebesar 0.9054 berarti model mampu mengklasifikasikan data uji dengan tingkat ketepatan sekitar 90.54 persen. Hasil ini menunjukkan bahwa model MLP memiliki performa yang baik dalam melakukan klasifikasi multi-kelas pada dataset digit tulisan tangan.

18. 

```
[18] ✓ 0s # VISUALISASI AKURASI & LOSS
plt.plot(history.history['accuracy'], label='Training Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.legend()
plt.show()

plt.plot(history.history['loss'], label='Training Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.legend()
plt.show()
```



Penjelasan :

Kode :

Kode ini digunakan untuk memvisualisasikan performa model selama proses training. Grafik pertama menampilkan perbandingan nilai akurasi data latih dan data validasi pada setiap epoch. Grafik kedua menampilkan perbandingan nilai loss data latih dan data validasi. Data yang digunakan berasal dari objek history yang dihasilkan saat proses training model.

Output :

Output berupa dua grafik, yaitu grafik akurasi dan grafik loss.

Grafik akurasi menunjukkan bahwa akurasi training dan validation meningkat secara konsisten seiring bertambahnya epoch. Nilai akurasi validation mengikuti tren akurasi training dan mencapai sekitar 90 persen, menandakan bahwa model mampu melakukan generalisasi dengan baik pada data yang belum pernah dilihat.

Grafik loss menunjukkan bahwa nilai loss pada data training dan validation menurun secara bertahap. Nilai loss validation cenderung stabil pada epoch akhir dan tidak meningkat secara signifikan, yang menandakan bahwa model tidak mengalami overfitting yang berlebihan. Pola ini juga mendukung keputusan penggunaan Early Stopping pada proses training.

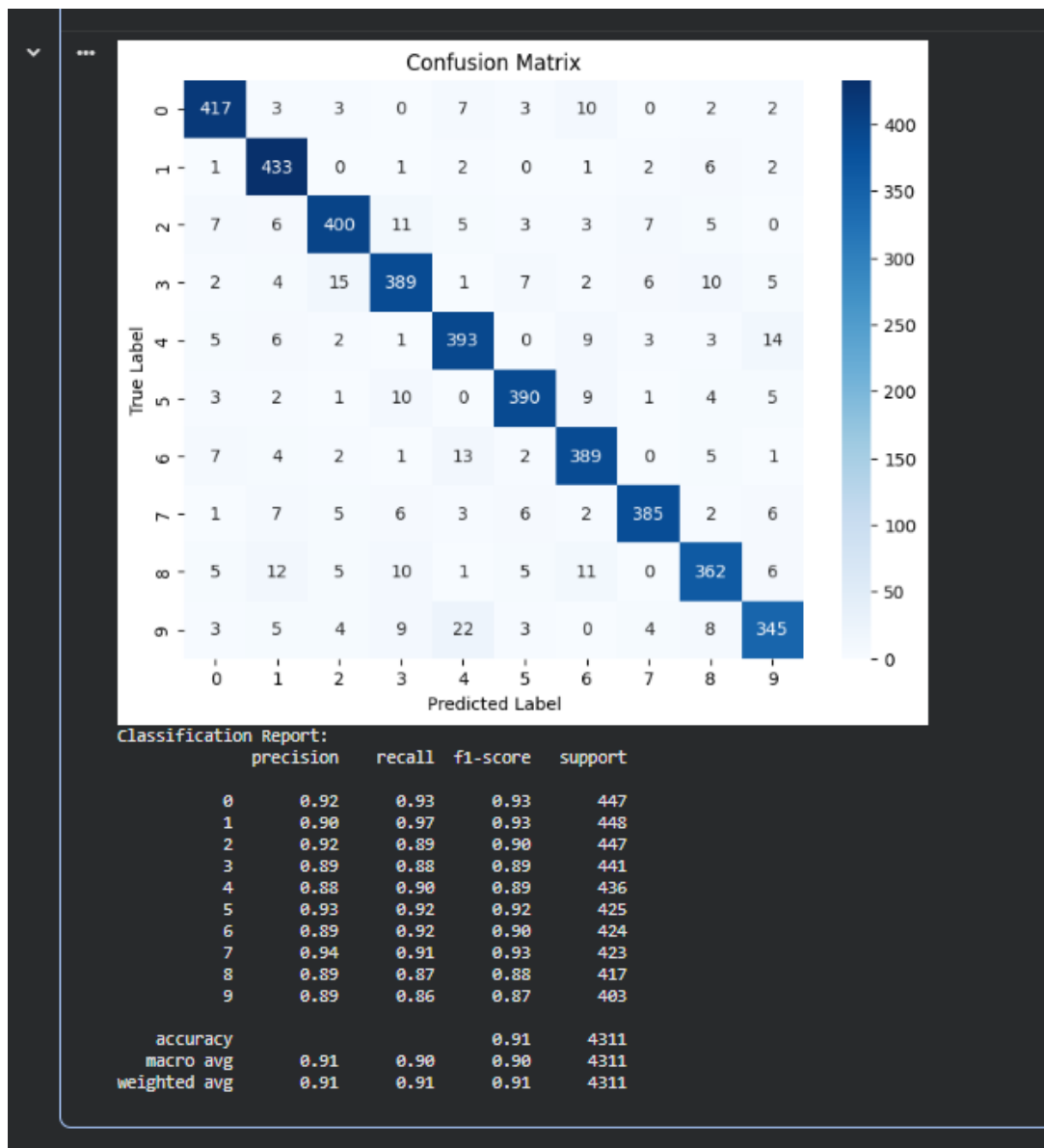
19.

```
[19] ✓ 2s
from sklearn.metrics import confusion_matrix, classification_report
import seaborn as sns
import matplotlib.pyplot as plt

# Confusion Matrix
cm = confusion_matrix(y_test, y_pred)

plt.figure(figsize=(8, 6))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
plt.xlabel('Predicted Label')
plt.ylabel('True Label')
plt.title('Confusion Matrix')
plt.show()

# Classification Report
print("Classification Report:")
print(classification_report(y_test, y_pred))
```



Penjelasan :

Kode :

Kode ini digunakan untuk melakukan evaluasi lanjutan terhadap performa model klasifikasi MLP.

- confusion_matrix digunakan untuk membandingkan label sebenarnya dengan label hasil prediksi model.
- heatmap digunakan untuk memvisualisasikan confusion matrix agar lebih mudah dianalisis.
- classification_report digunakan untuk menampilkan metrik evaluasi seperti precision, recall, dan f1-score pada setiap kelas digit.

Output :

- Confusion matrix menunjukkan bahwa sebagian besar prediksi berada pada diagonal utama, yang berarti model mampu mengklasifikasikan digit dengan benar.

- Kesalahan prediksi terlihat pada beberapa kelas, namun jumlahnya relatif kecil dibandingkan prediksi yang benar.
- Classification report menunjukkan nilai precision, recall, dan f1-score yang tinggi pada hampir semua kelas digit.
- Nilai akurasi keseluruhan sekitar 91 persen menandakan model memiliki performa klasifikasi yang baik dan stabil.

Kesimpulan :

Berdasarkan hasil implementasi dan evaluasi yang telah dilakukan, dapat disimpulkan bahwa model Artificial Neural Network (ANN) tipe Multilayer Perceptron (MLP) berhasil melakukan klasifikasi citra angka tulisan tangan dengan performa yang baik. Tahapan preprocessing seperti normalisasi, inversi citra, dan pembagian data secara stratified terbukti berperan penting dalam meningkatkan akurasi model. Proses pelatihan menunjukkan tren peningkatan akurasi dan penurunan loss yang stabil, serta tidak mengalami overfitting berkat penggunaan Batch Normalization, Dropout, dan Early Stopping. Hasil evaluasi akhir menunjukkan akurasi sebesar 90,54% pada data uji. Selain itu, confusion matrix dan classification report memperlihatkan bahwa model mampu mengenali hampir seluruh kelas dengan cukup konsisten. Dengan demikian, model yang dibangun telah memenuhi tujuan praktikum dan layak digunakan sebagai model klasifikasi dasar untuk pengenalan angka tulisan tangan.

Link GitHub :

1. Praktikum Mandiri :

[Sem3ML_Praktikum6/Praktikum13/13PrakMANDIRI.ipynb at main · Aisyaramli15/Sem3ML_Praktikum6](#)

2. Praktikum di Kelas :

- Bareng Asdos : [Sem3ML_Praktikum6/Praktikum13/13Prakbarengasdos.ipynb at main · Aisyaramli15/Sem3ML_Praktikum6](#)
- Slide PPT : [Sem3ML_Praktikum6/Praktikum13/13Prakdislide.ipynb at main · Aisyaramli15/Sem3ML_Praktikum6](#)